

Experiment ‘p_13_10_recursive_area_polygon_array’ Results

June 5, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 1

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 4

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 3

Problem Specification

Problem name: p_13_10_recursive_area_polygon_array

Natural language statement: Write a recursive method that takes an array of points of a polygon and returns its area. Cut off a triangle and use the fact that a triangle with corners (x_1, y_1) , (x_2, y_2) , (x_3, y_3) has area $(\text{abs}(x_1y_2 + x_2y_3 + x_3y_1 - y_1x_2 - y_2x_3 - y_3x_1)) / 2$.

Method signature: p_13_10_recursive_area_polygon_array(points: array<(real, real)>) returns (area: real)

Ensures

- `area == polygon_area(points[..])`

Requires

- `points.Length >= 3`

Functional Code Given

```
function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
```

```
var (x2, y2) := points[1];  
var (x3, y3) := points[2];  
var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);  
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;  
triangle_area + polygon_area([points[0]] + points[2..])  
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a recursive method that takes an array of points of a polygon and returns its area.

- Cut off a triangle and use the fact that a triangle with corners (x_1, y_1) , (x_2, y_2) , (x_3, y_3) has area $(\text{abs}(x_1y_2 + x_2y_3 + x_3y_1 - y_1x_2 - y_2x_3 - y_3x_1)) / 2$.

The signature should be:

`p_13_10_recursive_area_polygon_array(points: array<(real, real)>) returns (area: real)`

The method should respect the following contract:

`requires points.Length >= 3, ensures area == polygon_area(points[..])`

The contract uses the following dafny code:

```
function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;
    triangle_area + polygon_area([points[0]] + points[2..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that

- surround it. Do not show the signature nor contract. You must not use 'assume' nor
- 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnAzkjLrO7L0dmnk718lviKV17ZGE

Model: gpt-5.5-2026-04-23

Created at: 1780612708

Finish reason: stop

Usage: 1058 tokens in, and 6415 tokens out

```
{
  var (x1, y1) := points[0];
  var (x2, y2) := points[1];
  var (x3, y3) := points[2];
  var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
  var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

  assert points[..][0] == points[0];
  assert points[..][1] == points[1];
  assert points[..][2] == points[2];

  if points.Length == 3 {
    area := triangle_area;
  } else {
    assert points.Length > 3;

    var rest := new (real, real)[points.Length - 1];
    rest[0] := points[0];

    var i := 2;
    while i < points.Length
      invariant 2 <= i <= points.Length
      invariant rest.Length == points.Length - 1
      invariant rest[0] == points[0]
      invariant forall j :: 1 <= j < i - 1 ==> rest[j] == points[j + 1]
    {
      rest[i - 1] := points[i];
      i := i + 1;
    }

    assert i == points.Length;
    assert forall j :: 1 <= j < rest.Length ==> rest[j] == points[j + 1];

    assert |rest[..]| == |[points[0]] + points[2..]|;
    assert forall j :: 0 <= j < |rest[..]| ==> rest[..][j] == ([points[0]] + points[2..])[
      ↪ j] by {
      assert rest[..][j] == rest[j];
      if j == 0 {
        assert rest[j] == points[0];
        assert ([points[0]] + points[2..])[j] == points[0];
      } else {
        assert 1 <= j < rest.Length;
        assert rest[j] == points[j + 1];
      }
    }
  }
}
```

```

    assert 0 <= j - 1;
    assert j - 1 < |points[2..]|;
    assert ([points[0]] + points[2..])[j] == points[2..][j - 1];
    assert points[2..][j - 1] == points[2 + (j - 1)];
    assert 2 + (j - 1) == j + 1;
  }
}
assert rest[..] == [points[0]] + points[2..];
assert points[..][2..] == points[2..];
assert [points[..][0]] + points[..][2..] == [points[0]] + points[2..];

var subArea := p_13_10_recursive_area_polygon_array(rest);
area := triangle_area + subArea;
}
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(59,22): Error: unresolved
  ↪ identifier: j

```

```

|
59 | assert rest[..][j] == rest[j];
| ^

```

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(59,33): Error: unresolved
  ↪ identifier: j

```

```

|
59 | assert rest[..][j] == rest[j];
| ^

```

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(60,9): Error: unresolved
  ↪ identifier: j

```

```

|
60 | if j == 0 {
| ^

```

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(61,20): Error: unresolved
  ↪ identifier: j

```

```

|
61 | assert rest[j] == points[0];
| ^

```

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(62,43): Error: unresolved
  ↪ identifier: j

```

```

|
62 | assert ([points[0]] + points[2..])[j] == points[0];
| ^

```

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(64,20): Error: unresolved
  ↳ identifier: j
  |
64 | assert 1 <= j < rest.Length;
  | ^

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(65,20): Error: unresolved
  ↳ identifier: j
  |
65 | assert rest[j] == points[j + 1];
  | ^

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(65,33): Error: unresolved
  ↳ identifier: j
  |
65 | assert rest[j] == points[j + 1];
  | ^

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(66,20): Error: unresolved
  ↳ identifier: j
  |
66 | assert 0 <= j - 1;
  | ^

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(67,15): Error: unresolved
  ↳ identifier: j
  |
67 | assert j - 1 < |points[2..]|;
  | ^

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(68,43): Error: unresolved
  ↳ identifier: j
  |
68 | assert ([points[0]] + points[2..])[j] == points[2..][j - 1];
  | ^

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(68,61): Error: unresolved
  ↳ identifier: j
  |
68 | assert ([points[0]] + points[2..])[j] == points[2..][j - 1];
  | ^

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(69,27): Error: unresolved
  ↳ identifier: j
  |
69 | assert points[2..][j - 1] == points[2 + (j - 1)];
  | ^

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(69,49): Error: unresolved
  ↳ identifier: j
  |

```

```

69 | assert points[2..][j - 1] == points[2 + (j - 1)];
    | ^

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(70,20): Error: unresolved
  ↪ identifier: j
    |
70 | assert 2 + (j - 1) == j + 1;
    | ^

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(70,30): Error: unresolved
  ↪ identifier: j
    |
70 | assert 2 + (j - 1) == j + 1;
    | ^

16 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
  ↪ Dafny body, including the curly braces that surround it. Do not show the signature
  ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnB1pKczgq6I4PQ5Yr3vTshemA5lL

Model: gpt-5.5-2026-04-23

Created at: 1780612837

Finish reason: stop

Usage: 3047 tokens in, and 1756 tokens out

```

{
  var (x1, y1) := points[0];
  var (x2, y2) := points[1];
  var (x3, y3) := points[2];
  var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
  var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

  assert points[..][0] == points[0];
  assert points[..][1] == points[1];
  assert points[..][2] == points[2];

  if points.Length == 3 {
    area := triangle_area;
  } else {
    assert points.Length > 3;

    var rest := new (real, real)[points.Length - 1];
    rest[0] := points[0];

    var i := 2;
    while i < points.Length

```

```

    invariant 2 <= i <= points.Length
    invariant rest.Length == points.Length - 1
    invariant rest[0] == points[0]
    invariant forall j :: 1 <= j < i - 1 ==> rest[j] == points[j + 1]
  {
    rest[i - 1] := points[i];
    i := i + 1;
  }

  assert i == points.Length;
  assert forall j :: 1 <= j < rest.Length ==> rest[j] == points[j + 1];

  assert |rest[..]| == |[points[0]] + points[2..]|;
  forall j | 0 <= j < |rest[..]|
    ensures rest[..][j] == ([points[0]] + points[2..])[j]
  {
    assert rest[..][j] == rest[j];
    if j == 0 {
      assert rest[j] == points[0];
      assert ([points[0]] + points[2..])[j] == points[0];
    } else {
      assert 1 <= j < rest.Length;
      assert rest[j] == points[j + 1];
      assert 0 <= j - 1;
      assert j - 1 < |points[2..]|;
      assert ([points[0]] + points[2..])[j] == points[2..][j - 1];
      assert points[2..][j - 1] == points[2 + (j - 1)];
      assert 2 + (j - 1) == j + 1;
    }
  }
  assert rest[..] == [points[0]] + points[2..];
  assert points[..][2..] == points[2..];
  assert [points[..][0]] + points[..][2..] == [points[0]] + points[2..];

  var subArea := p_13_10_recursive_area_polygon_array(rest);
  area := triangle_area + subArea;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(79,55): Error: cannot prove
  ↪ termination; try supplying a decreases clause
|
79 | var subArea := p_13_10_recursive_area_polygon_array(rest);
| ^

```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
→ Dafny body, including the curly braces that surround it. Do not show the signature
→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnB2ZEC8tpvLJjrx6MBGjWz91s5Y

Model: gpt-5.5-2026-04-23

Created at: 1780612883

Finish reason: stop

Usage: 3166 tokens in, and 5696 tokens out

```
{
  var total: real := 0.0;
  var k: int := 1;

  assert points.Length >= 3;
  assert 1 <= points.Length - 2;

  assert |[points[0]] + points[1..]| == |points[..]|;
  forall j | 0 <= j < |points[..]|
    ensures ([points[0]] + points[1..])[j] == points[..][j]
  {
    if j == 0 {
      assert ([points[0]] + points[1..])[j] == points[0];
      assert points[..][j] == points[0];
    } else {
      assert 0 <= j - 1 < |points[1..]|;
      assert ([points[0]] + points[1..])[j] == points[1..][j - 1];
      assert points[1..][j - 1] == points[j];
      assert points[..][j] == points[j];
    }
  }
  assert [points[0]] + points[1..] == points[..];

  while k < points.Length - 2
    invariant 1 <= k <= points.Length - 2
    invariant total + polygon_area([points[0]] + points[k..]) == polygon_area(points[..])
    decreases points.Length - 2 - k
  {
    var oldTail := [points[0]] + points[k..];
    assert |oldTail| > 3;
    assert oldTail[0] == points[0];
    assert oldTail[1] == points[k];
    assert oldTail[2] == points[k + 1];

    var (x1, y1) := points[0];
    var (x2, y2) := points[k];
    var (x3, y3) := points[k + 1];
```

```

var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var tri: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

var newTail := [points[0]] + points[k + 1..];

assert |oldTail[2..]| == |points[k + 1..]|;
forall j | 0 <= j < |oldTail[2..]|
  ensures oldTail[2..][j] == points[k + 1..][j]
{
  assert oldTail[2..][j] == oldTail[j + 2];
  assert oldTail[j + 2] == points[k..][j + 1];
  assert points[k..][j + 1] == points[k + j + 1];
  assert points[k + 1..][j] == points[k + 1 + j];
}
assert oldTail[2..] == points[k + 1..];
assert [oldTail[0]] + oldTail[2..] == newTail;
assert polygon_area(oldTail) == tri + polygon_area(newTail);
assert total + polygon_area(oldTail) == polygon_area(points[..]);
assert total + tri + polygon_area(newTail) == polygon_area(points[..]);

total := total + tri;
k := k + 1;
assert [points[0]] + points[k..] == newTail;
assert total + polygon_area([points[0]] + points[k..]) == polygon_area(points[..]);
}

assert k == points.Length - 2;
var tail := [points[0]] + points[k..];
assert |tail| == 3;
assert tail[0] == points[0];
assert tail[1] == points[k];
assert tail[2] == points[k + 1];

var (x1, y1) := points[0];
var (x2, y2) := points[k];
var (x3, y3) := points[k + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var tri: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

assert polygon_area(tail) == tri;
area := total + tri;
assert area == polygon_area(points[..]);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(77,33): Error: assertion
  ↪ might not hold
  |

```

```
77 | assert polygon_area(oldTail) == tri + polygon_area(newTail);
    | ^^
```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the

→ Dafny body, including the curly braces that surround it. Do not show the signature

→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnB4TTzmhdk2basZsae3IQbIephXz

Model: gpt-5.5-2026-04-23

Created at: 1780613001

Finish reason: stop

Usage: 4375 tokens in, and 3205 tokens out

```
{
  var total: real := 0.0;
  var k: int := 1;

  assert |[points[0]] + points[1..]| == |points[..]|;
  forall j | 0 <= j < |points[..]|
    ensures ([points[0]] + points[1..])[j] == points[..][j]
  {
    if j == 0 {
      assert ([points[0]] + points[1..])[j] == points[0];
      assert points[..][j] == points[0];
    } else {
      assert 0 <= j - 1 < |points[1..]|;
      assert ([points[0]] + points[1..])[j] == points[1..][j - 1];
      assert points[1..][j - 1] == points[j];
      assert points[..][j] == points[j];
    }
  }
}
assert [points[0]] + points[1..] == points[..];

while k < points.Length - 2
  invariant 1 <= k <= points.Length - 2
  invariant total + polygon_area([points[0]] + points[k..]) == polygon_area(points[..])
  decreases points.Length - 2 - k
{
  var oldTail := [points[0]] + points[k..];
  assert |oldTail| > 3;
  assert oldTail[0] == points[0];
  assert oldTail[1] == points[k];
  assert oldTail[2] == points[k + 1];

  var (sx1, sy1) := oldTail[0];
```

```

var (sx2, sy2) := oldTail[1];
var (sx3, sy3) := oldTail[2];
var ssign: real := sx1 * (sy2 - sy3) + sx2 * (sy3 - sy1) + sx3 * (sy1 - sy2);
var stri: real := if ssign < 0.0 then -ssign / 2.0 else ssign / 2.0;

var newTail := [points[0]] + points[k + 1..];

assert |oldTail[2..]| == |points[k + 1..]|;
forall j | 0 <= j < |oldTail[2..]|
  ensures oldTail[2..][j] == points[k + 1..][j]
{
  assert oldTail[2..][j] == oldTail[j + 2];
  assert 0 < j + 2 < |oldTail|;
  assert oldTail[j + 2] == points[k..][j + 1];
  assert 0 <= j + 1 < |points[k..]|;
  assert points[k..][j + 1] == points[k + j + 1];
  assert points[k + 1..][j] == points[k + 1 + j];
  assert k + j + 1 == k + 1 + j;
}
assert oldTail[2..] == points[k + 1..];
assert [oldTail[0]] + oldTail[2..] == newTail;

assert polygon_area(oldTail) == stri + polygon_area([oldTail[0]] + oldTail[2..]);
assert polygon_area(oldTail) == stri + polygon_area(newTail);
assert total + polygon_area(oldTail) == polygon_area(points[..]);
assert total + stri + polygon_area(newTail) == polygon_area(points[..]);

total := total + stri;
k := k + 1;
assert [points[0]] + points[k..] == newTail;
assert total + polygon_area([points[0]] + points[k..]) == polygon_area(points[..]);
}

assert k == points.Length - 2;
var tail := [points[0]] + points[k..];
assert |tail| == 3;
assert tail[0] == points[0];
assert tail[1] == points[k];
assert tail[2] == points[k + 1];

var (tx1, ty1) := tail[0];
var (tx2, ty2) := tail[1];
var (tx3, ty3) := tail[2];
var tsign: real := tx1 * (ty2 - ty3) + tx2 * (ty3 - ty1) + tx3 * (ty1 - ty2);
var ttri: real := if tsign < 0.0 then -tsign / 2.0 else tsign / 2.0;

assert polygon_area(tail) == ttri;
area := total + ttri;
assert area == polygon_area(points[..]);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```
problem_data/p_13_10_recursive_area_polygon_array/program.dfy(78,33): Error: assertion
  ↳ might not hold
```

```
|
78 | assert polygon_area(oldTail) == stri + polygon_area([oldTail[0]] + oldTail[2..]);
| ^^
```

```
problem_data/p_13_10_recursive_area_polygon_array/program.dfy(102,28): Error: assertion
  ↳ might not hold
```

```
|
102 | assert polygon_area(tail) == ttri;
| ^^
```

Dafny program verifier finished with 2 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
↳ Dafny body, including the curly braces that surround it. Do not show the signature
↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnB5ddhxH9BmRMNruiopfcjQLbkXS

Model: gpt-5.5-2026-04-23

Created at: 1780613073

Finish reason: length

Usage: 5775 tokens in, and 1282 tokens out

The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```
function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
```

```

    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;
    triangle_area + polygon_area([points[0]] + points[2..])
}

method p_13_10_recursive_area_polygon_array(points: array<(real, real)>) returns (area:
    ↪ real)
    requires points.Length >= 3
    ensures area == polygon_area(points[..])
{
    var total: real := 0.0;
    var k: int := 1;

    assert |[points[0]] + points[1..]| == |points[..|;
    forall j | 0 <= j < |points[..|
        ensures ([points[0]] + points[1..])[j] == points[..][j]
    {
        if j == 0 {
            assert ([points[0]] + points[1..])[j] == points[0];
            assert points[..][j] == points[0];
        } else {
            assert 0 <= j - 1 < |points[1..|;
            assert ([points[0]] + points[1..])[j] == points[1..][j - 1];
            assert points[1..][j - 1] == points[j];
            assert points[..][j] == points[j];
        }
    }
    assert [points[0]] + points[1..] == points[..];

    while k < points.Length - 2
        invariant 1 <= k <= points.Length - 2
        invariant total + polygon_area([points[0]] + points[k..]) == polygon_area(points[..])
        decreases points.Length - 2 - k
    {
        var oldTail := [points[0]] + points[k..];
        assert |oldTail| > 3;
        assert oldTail[0] == points[0];
        assert oldTail[1] == points[k];
        assert oldTail[2] == points[k + 1];

        var (sx1, sy1) := oldTail[0];
        var (sx2, sy2) := oldTail[1];
        var (sx3, sy3) := oldTail[2];
        var ssign: real := sx1 * (sy2 - sy3) + sx2 * (sy3 - sy1) + sx3 * (sy1 - sy2);
        var stri: real := if ssign < 0.0 then -ssign / 2.0 else ssign / 2.0;

        var newTail := [points[0]] + points[k + 1..];

        assert |oldTail[2..]| == |points[k + 1..|;

```

```

forall j | 0 <= j < |oldTail[2..]|
  ensures oldTail[2..][j] == points[k + 1..][j]
{
  assert oldTail[2..][j] == oldTail[j + 2];
  assert 0 < j + 2 < |oldTail|;
  assert oldTail[j + 2] == points[k..][j + 1];
  assert 0 <= j + 1 < |points[k..]|;
  assert points[k..][j + 1] == points[k + j + 1];
  assert points[k + 1..][j] == points[k + 1 + j];
  assert k + j + 1 == k + 1 + j;
}
assert oldTail[2..] == points[k + 1..];
assert [oldTail[0]] + oldTail[2..] == newTail;

assert polygon_area(oldTail) == stri + polygon_area([oldTail[0]] + oldTail[2..]);
assert polygon_area(oldTail) == stri + polygon_area(newTail);
assert total + polygon_area(oldTail) == polygon_area(points[..]);
assert total + stri + polygon_area(newTail) == polygon_area(points[..]);

total := total + stri;
k := k + 1;
assert [points[0]] + points[k..] == newTail;
assert total + polygon_area([points[0]] + points[k..]) == polygon_area(points[..]);
}

assert k == points.Length - 2;
var tail := [points[0]] + points[k..];
assert |tail| == 3;
assert tail[0] == points[0];
assert tail[1] == points[k];
assert tail[2] == points[k + 1];

var (tx1, ty1) := tail[0];
var (tx2, ty2) := tail[1];
var (tx3, ty3) := tail[2];
var tsign: real := tx1 * (ty2 - ty3) + tx2 * (ty3 - ty1) + tx3 * (ty1 - ty2);
var ttri: real := if tsign < 0.0 then -tsign / 2.0 else tsign / 2.0;

assert polygon_area(tail) == ttri;
area := total + ttri;
assert area == polygon_area(points[..]);
}

```

Total Token Usage

Input tokens: 17421

Output tokens: 18354

Reasoning tokens: 14537

Sum of 'total tokens': 35775

Experiment Timings

Overall Experiment started at 1780612707574, ended at 1780613102756, lasting 395182ms (395.18 seconds)

Iteration #4 started at 1780613000775, ended at 1780613073012, lasting 72237ms (72.24 seconds)

Iteration #5 started at 1780613073013, ended at 1780613102592, lasting 29579ms (29.58 seconds)

Iteration #1 started at 1780612707575, ended at 1780612836781, lasting 129206ms (129.21 seconds)

Iteration #2 started at 1780612836782, ended at 1780612883522, lasting 46740ms (46.74 seconds)

Iteration #3 started at 1780612883522, ended at 1780613000775, lasting 117253ms (117.25 seconds)